CI-Lib Core Primitives: Implementation Changelog

Jonas Hallgren
Equilibria Network

March 2026

Abstract

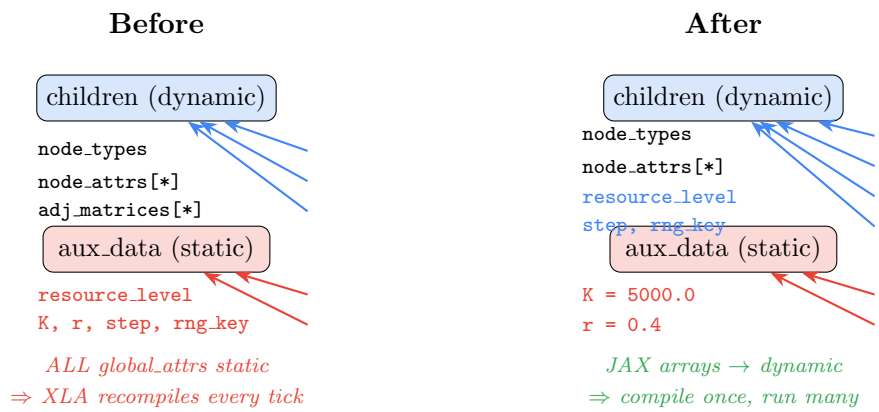
This document records the implementation of CI-Lib's four core primitives: GraphState (immutable typed state), Transform (pure functions with read/write declarations), Schedule (temporal orchestration), and Composition Operators (sequential, parallel, conditional). Each section includes architecture diagrams and code excerpts.

Contents

1 GraphState Pytree Fix

The foundational change: partitioning `global_attrs` so JAX arrays are traced (dynamic) and Python scalars are baked into the compiled kernel (static).

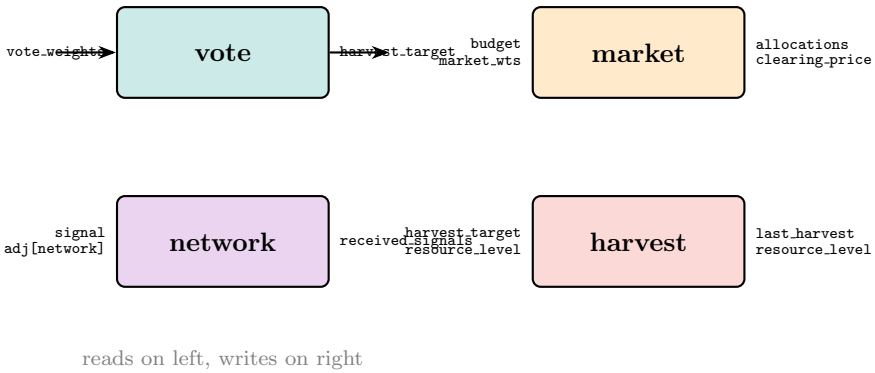
1.1 Before and After



Partitioning rule: `isinstance(v, (jnp.ndarray, jax.Array))` → children; everything else → aux_data. Impact: ∼3–10× speedup by eliminating per-tick recompilation.

2 Typed Transforms

The `@transform` decorator attaches read/write metadata to pure functions.

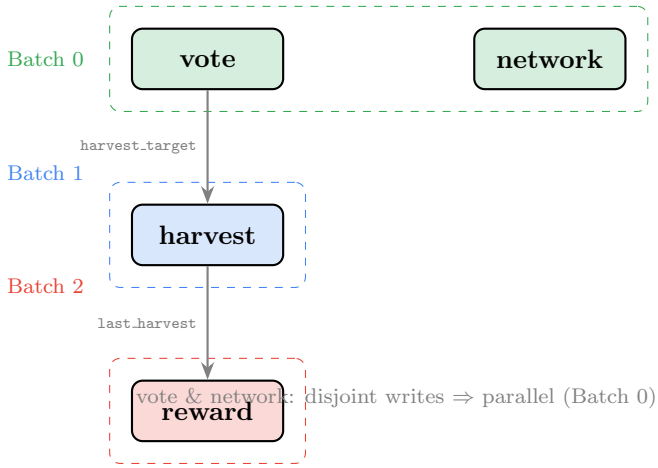



```
@transform(reads=["vote_weights"], writes=["harvest_target"])
def direct_democracy(state: GraphState) -> GraphState:
    ...
```

3 Pipeline Compiler

The compiler builds a dependency DAG from read/write sets, topologically sorts into parallel batches, and composes into a single Transform.

3.1 Dependency DAG



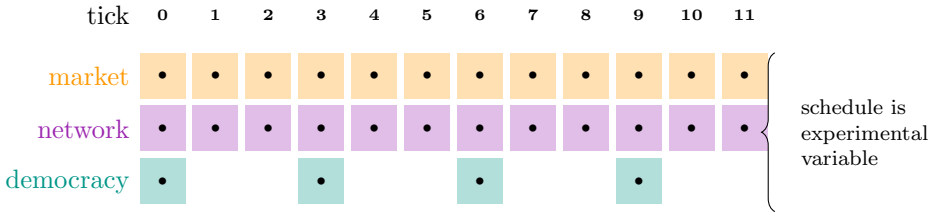
Algorithm: Kahn's topological sort. At each step, collect *all* zero-in-degree nodes into one batch. Transforms within a batch run via `parallel()`; batches execute via `sequential()`.

```
pipeline = compile_pipeline([vote, harvest, reward, network])
# Derived order: [vote || network] -> harvest -> reward
```

4 Schedule Primitive

The schedule maps clock ticks to active transforms. Each transform has a *cadence* c_i and *phase offset* ϕ_i: it fires at tick t iff $(t - \phi_i) \bmod c_i = 0$.

4.1 Timeline Diagram



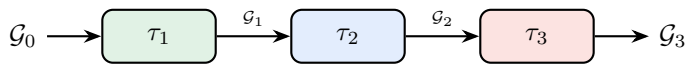
The schedule uses `jax.lax.cond` for JIT-compatible conditional execution: both branches are traced, only one executes at runtime.

```
entries = [
    ScheduleEntry(market, cadence=1),
    ScheduleEntry(democracy, cadence=k), # k is experimental
]
step = make_scheduled_step(entries)
```

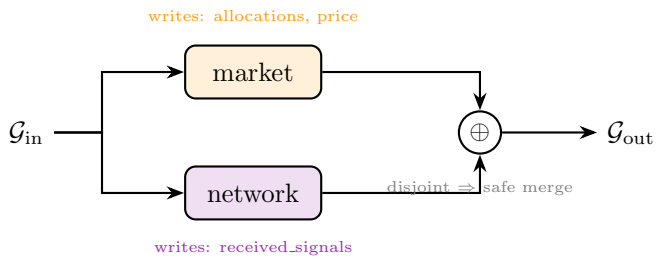
5 Composition Operators

Three operators, closed under composition (applying any operator to transforms produces a new transform $\mathcal{G} \to \mathcal{G}$).

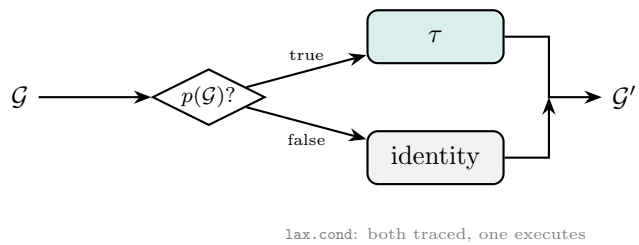
5.1 Sequential



5.2 Parallel



5.3 Conditional



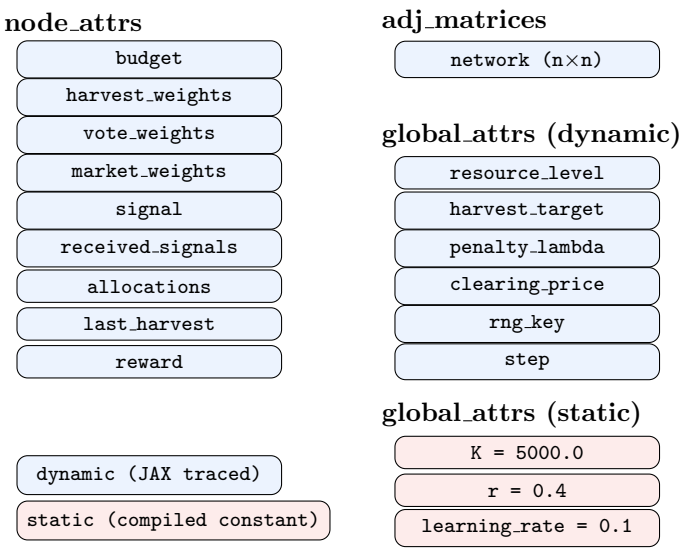
6 Type Contracts

Each mechanism type declares which GraphState fields it reads and writes. Disjoint write sets enable safe parallel composition.

Type	Reads	Writes	Topology
Market	budget, market_wts, resource	allocations, budget, price	Global hub
Network	signal, adj[network]	received_signals	Local sparse
Democracy	vote_wts, last_harvest	harvest_target, penalty_λ	All→one→all

Write disjointness: Market writes {allocations, budget, price}. Network writes {received_signals}. Democracy writes {target, λ}. Intersection $= \emptyset \Rightarrow$ `parallel(market, network, democracy)` is valid.

7 Fishing Commons State



8 Summary of Changes

File	Change	Status
`core/graph.py`	Pytree fix: partition global_attrs	Done
`core/category.py`	`@transform`, `parallel`, `conditional`	Done
`core/category.py`	Bug fixes: identity, sequential, compose metadata	Done
`core/pipeline.py`	Pipeline compiler (typed DAG)	Done
`core/schedule.py`	Schedule primitive (cadence + phase)	Done
`fishing_commons/state.py`	Initial state factory	Done
`mechanisms/__init__.py`	Type contracts	Done

Remaining: `lax.scan` episode runner, `vmap` over seeds, mechanism implementations (market, network, democracy), resource dynamics, reward, learning transforms, and the full 13,200-episode sweep.